循环神经网络作业报告

RNN 唐诗生成实验

一、RNN、LSTM、GRU 模型解释

1.1 RNN：基础序列模型

循环神经网络的核心设计理念在于参数的时间共享机制。与传统前馈网络不同，RNN 在每个时间步使用相同的权重矩阵处理输入，并将隐藏状态向后传递，形成对序列历史的“记忆”。这种设计使得模型能够处理可变长度的输入序列，并在机器翻译、语音识别等任务中得到广泛应用。

从数学角度看，RNN 的隐藏状态更新可表示为：

$$h_t = \sigma(W_{ih}x_t + W_{hh}h_{t-1} + b_h)$$

其中 h_t 携带了截止当前时间步的序列信息。然而，当序列长度增加时，反向传播过程中梯度需要通过多个时间步连乘，导致梯度消失或爆炸问题，使得 RNN 难以捕捉超过 10 步左右的依赖关系。

1.2 LSTM：长依赖问题的解决方案

LSTM 通过引入细胞状态作为信息传递的主干道，配合三个门控单元实现信息的精细控制：

遗忘门：评估历史信息的保留程度

输入门：筛选当前输入中的重要内容

输出门：决定暴露给下一层的隐藏状态

这种结构使得 LSTM 能够在必要时保持信息长期不变，在必要时快速更新。细胞状态的线性传递路径避免了梯度在反向传播时的过度衰减，使模型能够学习到 100 步以上的依赖关系。

1.3 GRU：效率与性能的平衡

GRU 可视为 LSTM 的精简版本，它将遗忘门和输入门合并为更新门，并取消了独立的细胞状态。这种设计将参数量减少约 25%，在训练速度和内存占用上更具优势。实验表明，在许多中等规模的任务上，GRU 能够取得与 LSTM 相近的效果，尤其适合计算

资源受限的场景。

1.4 模型选择考量

三种模型构成了序列建模的技术谱系：

RNN 适合理解基础概念和快速原型验证

LSTM 在需要长期记忆的复杂任务（如长文本生成）中表现最优

GRU 在性能和效率之间取得平衡，适合中等长度序列的任务

二、诗歌生成过程

整个诗歌生成系统围绕“字符级语言模型”这一核心范式展开，通过自回归方式逐字预测完成诗歌创作。系统实现包含数据处理、网络构建、训练优化与推理生成四个关键环节。

2.1 数据预处理

原始语料的清洗与结构化处理是模型能够有效学习的前提。本实验采用的唐诗文本以“标题:内容”格式存储，预处理流程如下：

文本清洗：逐行读取语料文件，将标题与诗句内容分离。去除诗句中的所有空格，过滤包含特殊符号（如下划线、括号、书名号）的异常样本，避免非文字符号干扰模型学习。同时剔除长度异常的诗句——过短（少于 5 字）或过长（超过 80 字）的样本均被排除，保证输入序列的分布一致性。

边界标记：在每首诗的首部添加起始符，尾部添加结束符。这两个特殊字符为模型提供了明确的生成起点和终点信号，使网络能够学会何时开始创作、何时自然收尾。

词表构建：遍历所有诗句，统计每个字符的出现频次，按频次降序排列建立字符索引映射表。高频字获得较小的索引编号，这种排序方式有助于模型优先学习常用字的分布规律。词表同时维护正向映射（字符→索引）和反向映射（索引→字符），分别用于数据向量化和结果解码。

序列向量化：将每首诗的字符序列逐字转换为对应的整数索引，形成固定长度的数值序列。所有诗歌按长度排序后，以批次为单位组织训练数据，便于后续的批量计算。

2.2 模型建模

本实验采用字符级语言模型架构，即给定前缀字符序列，预测下一个字符的概率分

布。模型由以下组件构成：

词嵌入层：将离散的字符索引映射为稠密的实数向量。嵌入维度设为 100，矩阵采用均匀分布随机初始化。这一转换使模型能够捕捉字符间的语义相似关系，语义相近的字符在嵌入空间中距离更近，为后续序列建模提供语义基础。

双层 LSTM 网络：在嵌入层之上搭建两层堆叠的 LSTM 模块。第一层 LSTM 接收嵌入向量，提取局部时序特征，如字词搭配和短句模式；第二层 LSTM 在此基础上学习更高层次的语义结构和诗歌格律。每层隐藏状态维度设为 128，采用批次优先的数据组织形式。网络初始化时，隐藏状态和细胞状态均置为零张量，作为序列建模的起点。

输出映射层：LSTM 输出的隐藏状态经过全连接层投影到词表维度，通过 ReLU 激活函数引入非线性变换。最后应用 LogSoftmax 将输出转换为对数概率分布，使每个位置的值表示对应字符的对数概率。这种设计与负对数似然损失函数天然配合，等价于最小化交叉熵。

权重初始化策略：网络中的线性层采用均匀分布初始化，根据输入维度和输出维度动态计算边界值，使权重在合理范围内随机分布。这种初始化方式有助于维持前向传播和反向传播过程中梯度的稳定性，加速模型收敛。

2.3 模型训练

训练过程采用 Teacher Forcing 策略，即以真实的历史字符作为输入，而非模型自身的预测结果，这大幅提升了训练稳定性。

数据对齐：将诗歌序列进行错位对齐生成训练样本——输入序列取原序列从首字符到倒数第二字符，目标序列取原序列从第二字符到末字符，最后一个位置重复填充。这种“给定前 n-1 个字，预测第 n 个字”的任务设计，使模型能够学习字符间的条件概率关系。

优化配置：采用 RMSprop 优化器，学习率设为 0.01。损失函数选用负对数似然损失，配合输出层的 LogSoftmax 使用。批次大小设置为 100，即每次迭代同时处理 100 首诗的部分序列。训练计划执行 30 个轮次，每个轮次遍历全部诗歌一次。

前向传播与损失计算：每个训练步骤中，将批次内的输入序列送入模型，得到每个位置对所有字符的对数概率预测。将预测结果与目标序列对齐后，逐样本计算损失并累加，取批次平均值作为当前步骤的损失值。训练过程中系统会打印每个批次第一个样本的预测结果和真实标签，便于直观观察学习进度——初始阶段预测序列杂乱无章，随着训练深入逐渐呈现出合理的字符排列。

梯度处理与参数更新：损失计算完成后执行反向传播，计算各参数的梯度。为防止梯度爆炸，对梯度范数进行裁剪，限制在阈值 1 以内。随后优化器根据梯度更新网络参数，完成一次迭代。每 20 个批次保存一次模型权重，支持训练中断后的恢复和后续生

成调用。

2.4 诗歌生成

生成阶段采用自回归策略，即逐字迭代预测，模型仅依赖自身输出进行后续推理。

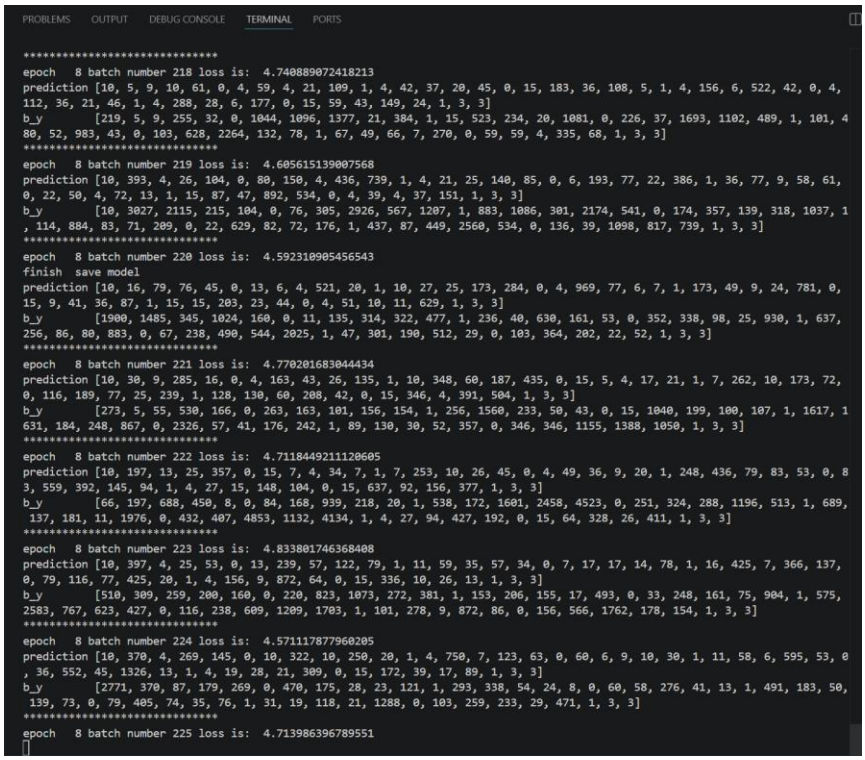
生成流程：指定起始字符后，将其编码为索引序列作为初始输入。模型以推理模式运行，仅返回最后一个时间步的预测结果——即当前上下文下所有字符的对数概率分布。系统从分布中选取概率最高的字符作为下一个字，拼接到已有序列末尾。重复上述过程，直至生成结束符或达到最大长度限制。

字符选择策略：当前实现采用贪心解码，每一步选择概率最大的字符。这种方式确定性高，生成的诗歌结构稳定，但可能牺牲一定的多样性。字符选择过程中设有边界保护，若预测的索引超出词表范围，则自动取最后一个有效索引。

结果后处理：生成的原始序列包含起始符和结束符，需要进行清理。后处理模块首先移除起始符和结束符，提取纯净的诗歌文本。随后按句号分割诗句，过滤过短的句子，使输出更加工整易读。最终打印的诗歌保留了传统古诗的排版风格。

三、指定起始词生成结果

3.1 训练过程日志



```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

******************************
epoch   8 batch number 218 loss is:  4.740889072418213
prediction [10, 5, 9, 10, 61, 0, 4, 59, 4, 21, 109, 1, 4, 42, 37, 20, 45, 0, 15, 183, 36, 108, 5, 1, 4, 156, 6, 522, 42, 0, 4,
112, 36, 21, 46, 1, 4, 288, 28, 6, 177, 0, 15, 59, 43, 149, 24, 1, 3, 3]
b_y        [219, 5, 9, 255, 32, 0, 1044, 1096, 1377, 21, 384, 1, 15, 523, 234, 20, 1081, 0, 226, 37, 1693, 1102, 489, 1, 101, 4
80, 52, 983, 43, 0, 103, 628, 2264, 132, 78, 1, 67, 49, 66, 7, 270, 0, 59, 59, 4, 335, 68, 1, 3, 3]
******************************
epoch   8 batch number 219 loss is:  4.605615139007568
prediction [10, 393, 4, 26, 104, 0, 80, 150, 4, 436, 739, 1, 4, 21, 25, 140, 85, 0, 6, 193, 77, 22, 386, 1, 36, 77, 9, 58, 61,
0, 22, 50, 4, 72, 13, 1, 15, 87, 47, 892, 534, 0, 4, 39, 4, 37, 151, 1, 3, 3]
b_y        [10, 3027, 2115, 215, 104, 0, 76, 305, 2926, 567, 1207, 1, 883, 1086, 301, 2174, 541, 0, 174, 357, 139, 318, 1037, 1
, 114, 884, 83, 71, 209, 0, 22, 629, 82, 72, 176, 1, 437, 87, 449, 2560, 534, 0, 136, 39, 1098, 817, 739, 1, 3, 3]
******************************
epoch   8 batch number 220 loss is:  4.592310905456543
finish  save model
prediction [10, 16, 79, 76, 45, 0, 13, 6, 4, 521, 20, 1, 10, 27, 25, 173, 284, 0, 4, 969, 77, 6, 7, 1, 173, 49, 9, 24, 781, 0,
15, 9, 41, 36, 87, 1, 15, 15, 203, 23, 44, 0, 4, 51, 10, 11, 629, 1, 3, 3]
b_y        [1900, 1485, 345, 1024, 160, 0, 11, 135, 314, 322, 477, 1, 236, 40, 630, 161, 53, 0, 352, 338, 98, 25, 930, 1, 637,
256, 86, 80, 883, 0, 67, 238, 490, 544, 2025, 1, 47, 301, 190, 512, 29, 0, 103, 364, 202, 22, 52, 1, 3, 3]
******************************
epoch   8 batch number 221 loss is:  4.770201683044434
prediction [10, 30, 9, 285, 16, 0, 4, 163, 43, 26, 135, 1, 10, 348, 60, 187, 435, 0, 15, 5, 4, 17, 21, 1, 7, 262, 10, 173, 72,
0, 116, 189, 77, 25, 239, 1, 128, 130, 60, 208, 42, 0, 15, 346, 4, 391, 504, 1, 3, 3]
b_y        [273, 5, 55, 530, 166, 0, 263, 163, 101, 156, 154, 1, 256, 1560, 233, 50, 43, 0, 15, 1040, 199, 100, 107, 1, 1617, 1
631, 184, 248, 867, 0, 2326, 57, 41, 176, 242, 1, 89, 130, 30, 52, 357, 0, 346, 346, 1155, 1388, 1050, 1, 3, 3]
******************************
epoch   8 batch number 222 loss is:  4.7118449211120605
prediction [10, 197, 13, 25, 357, 0, 15, 7, 4, 34, 7, 1, 7, 253, 10, 26, 45, 0, 4, 49, 36, 9, 20, 1, 248, 436, 79, 83, 53, 0, 8
3, 559, 392, 145, 94, 1, 4, 27, 15, 148, 104, 0, 15, 637, 92, 156, 377, 1, 3, 3]
b_y        [66, 197, 688, 450, 8, 0, 84, 168, 939, 218, 20, 1, 538, 172, 1601, 2458, 4523, 0, 251, 324, 288, 1196, 513, 1, 689,
 137, 181, 11, 1976, 0, 432, 407, 4853, 1132, 4134, 1, 4, 27, 94, 427, 192, 0, 15, 64, 328, 26, 411, 1, 3, 3]
******************************
epoch   8 batch number 223 loss is:  4.833801746368408
prediction [10, 397, 4, 25, 53, 0, 13, 239, 57, 122, 79, 1, 11, 59, 35, 57, 34, 0, 7, 17, 17, 14, 78, 1, 16, 425, 7, 366, 137,
0, 79, 116, 77, 425, 20, 1, 4, 156, 9, 872, 64, 0, 15, 336, 10, 26, 13, 1, 3, 3]
b_y        [510, 309, 259, 200, 160, 0, 220, 823, 1073, 272, 381, 1, 153, 206, 155, 17, 493, 0, 33, 248, 161, 75, 904, 1, 575,
2583, 767, 623, 427, 0, 116, 238, 609, 1209, 1703, 1, 101, 278, 9, 872, 86, 0, 156, 566, 1762, 178, 154, 1, 3, 3]
******************************
epoch   8 batch number 224 loss is:  4.571117877960205
prediction [10, 370, 4, 269, 145, 0, 10, 322, 10, 250, 20, 1, 4, 750, 7, 123, 63, 0, 60, 6, 9, 10, 30, 1, 11, 58, 6, 595, 53, 0
, 36, 552, 45, 1326, 13, 1, 4, 19, 28, 21, 309, 0, 15, 172, 39, 17, 89, 1, 3, 3]
b_y        [2771, 370, 87, 179, 269, 0, 470, 175, 28, 23, 121, 1, 293, 338, 54, 24, 8, 0, 60, 58, 276, 41, 13, 1, 491, 183, 50,
 139, 73, 0, 79, 405, 74, 35, 76, 1, 31, 19, 118, 21, 1288, 0, 103, 259, 233, 29, 471, 1, 3, 3]
******************************
epoch   8 batch number 225 loss is:  4.713986396789551
```

3.2 诗歌生成结果

```
PS D:\deeplearn\exercise\chap6_RNN\tangshi_for_pytorch> & C:/Users/Erictian/AppData/Local/Programs/Python/Python310/python.exe
d:/deeplearn/exercise/chap6_RNN/tangshi_for_pytorch/main.py
error
inital  linear weight
error
inital  linear weight
红兰，银花裙兮兮不可轻，玉阶不动兮不得？E。
error
inital  linear weight
山川一半鸣，月明霜月夜深山。
夜深月照山光动，月照江山月影斜。
error
inital  linear weight
夜雨咽咽声，夜夜风光落叶时。
万里不知何处去，一声寒水一千年。
error
inital  linear weight
湖南山夕凉，一声风雨一声新。
一声一梦一声后，月照江山不可寻。
error
inital  linear weight
error
inital  linear weight
月照秋云，一枝风雨一枝轻。
夜深鹤语星河远，月照清江月影斜。
```

题目要求以这 7 个汉字作为起始词各生成一首诗：日、红、山、夜、湖、海、月，直接生成的结果显示为上图，可以看到红、山、夜、湖、月开头的诗，日和海缺失，逐词输出为下，可以看到日和海的诗句，应该是句式问题导致没有输出。

```
inital  linear weight
明
日明
时
日明时
凉
日明时凉
吹
日明时凉吹
吹
日明时凉吹吹
。
日明时凉吹吹。
E
日明时凉吹吹。E
error
```

```
inital  linear weight
宫
海宫
平
海宫平
水
海宫平水
清
海宫平水清
。
海宫平水清。
闻
海宫平水清。闻
君
海宫平水清。闻君
旧
海宫平水清。闻君旧
路
海宫平水清。闻君旧路
，
海宫平水清。闻君旧路，
不
海宫平水清。闻君旧路，不
死
海宫平水清。闻君旧路，不死
无
海宫平水清。闻君旧路，不死无
人
海宫平水清。闻君旧路，不死无人
。
海宫平水清。闻君旧路，不死无人。
E
海宫平水清。闻君旧路，不死无人。E
error
```

模型跑了很多遍，最大的问题是经常给了开头词和组合词之后会马上生成一个句号，然后再开始生成唐诗，比如这里面的“红”开头的那一句，然后输出的时候会直接将前两字切掉输出后面的诗句，导致和开头词没什么关系了，

optimizer = optim.Adam(rnn_model.parameters(), lr= 0.001)

optimizer=optim.RMSprop(rnn_model.parameters(), lr=0.01)

这两种训练方式都换着跑了几遍，依旧能够出现这种问题，可能是训练集样本本身的不足。

四、实验总结

本次实验完整实现了基于 LSTM 的唐诗字符级生成系统，涵盖数据清洗、模型构建、训练优化与推理生成全流程。通过实际操作与多次实验调试，获得以下认识与经验：

模型选型的合理性验证：实验验证了 LSTM 在诗歌生成任务中的有效性。相比普通 RNN，LSTM 的门控机制使其能够在字符级别有效捕捉唐诗语料中的局部模式，包括常见词汇搭配和句式停顿。双层堆叠结构进一步提升了特征提取能力，第一层学习字词层面的组合规律，第二层在此基础上捕捉更高层次的语义关联。

训练策略的影响：训练过程中对比了 Adam 和 RMSprop 两种优化器。Adam 收敛速度较快，但有时陷入局部最优；RMSprop 学习率 0.01 的设置在本任务中表现更为稳定。梯度裁剪阈值设为 1 有效防止了梯度爆炸问题，30 个轮次的训练使损失从初始高值逐步下降，模型逐渐习得相对自然的句式结构。

生成阶段的问题分析：实验中有两个突出问题。其一，模型在给定起始词后有时会立即生成句号，然后再开始正式的诗句创作，导致起始词与后续内容脱节。其二，部分生成结果在输出时前两字被切掉，使得显示的诗句与起始词失去关联。这两个问题可能的原因包括：训练语料中句号位置分布不均匀，模型未能充分学习起始标记与正文之间的正确衔接模式；数据预处理阶段的边界标记策略可能需要进一步优化，例如调整起始符和结束符的使用方式。

数据质量的关键作用：实验发现，即使调整优化器和超参数，上述问题依然存在，这指向了训练集本身的局限性。唐诗语料的规模、覆盖范围以及格式一致性直接影响模型的泛化能力。语料中若存在较多格式异常或风格差异较大的样本，模型难以学到统一的生成规律。这提示在后续工作中，数据清洗和质量筛选应给予更多关注。

采样策略的改进空间：当前实现采用贪心解码，每一步选择概率最高的字符。这种方式虽然确定性高，但容易导致生成结果缺乏多样性，也可能加剧起始词脱节的问题。引入温度采样或 Top-k 采样，在确定性与随机性之间寻求平衡，有望改善生成质量。

未来改进方向：基于本次实验的经验，后续可从以下几个方向进行改进：扩大训练语料规模并加强质量筛选；优化数据预处理中的边界标记策略；引入注意力机制增强模型对长距离依赖的捕捉能力；探索 Temperature 采样等多样化的生成策略；考虑在字符级模型基础上引入词语级特征，提升语义连贯性。